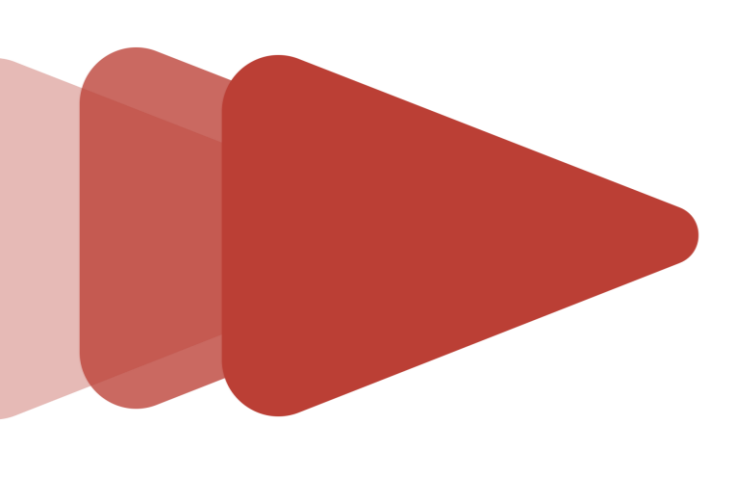




Estándar de Desarrollo Aplicaciones Web – VUE JS

NAC-PTITARE24- 00000005

Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información - Datos
Dirección Nacional Tecnología

Fecha de Actualización: Agosto 2024
Versión: 2.0.0

Cláusula de Responsabilidad

Los firmantes de este documento, en representación de sus unidades administrativas a las que pertenecen, aceptan y aprueban el presente documento y a su vez se comprometen a respaldar su cumplimiento, hacer difusión del mismo en sus respectivas unidades y a sugerir mejoras para versiones futuras en el caso de que se las identifiquen.

Fecha de Vigencia 05/08/2024	Dirección Nacional de Tecnología	VERSIÓN 2.0.0	PÁGINAS 26
Estándar de Desarrollo Aplicaciones Web – VUE JS			
	CARGO	FIRMA	
APROBADO POR:	Ing. Paulina Correa Aguayo DIRECTORA NACIONAL DE TECNOLOGÍA (S)	 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
REVISADO POR:	Ing. Paulina Correa Aguayo JEFE NACIONAL DEPARTAMENTO PLANIFICACIÓN TI Y GESTIÓN INFORMACIÓN - DATOS	 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
	Ing. Alexandra Pasquel Pintado JEFE NACIONAL DEPARTAMENTO DESARROLLO (S)	 Firmado electrónicamente por: ALEXANDRA ELIZABETH PASQUEL PINTADO	
	Ing. José Cerón Villamagua JEFE NACIONAL DEPARTAMENTO SEGURIDAD INFORMÁTICA	 Firmado electrónicamente por: JOSE FELIX CERON VILLAMAGUA	
	Ing. Catalina Maribel Chico Urbina COORDINADOR NACIONAL DESARROLLO Y MANTENIMIENTO (S)	 Firmado electrónicamente por: CATALINA MARIBEL CHICO URBINA	
	Ing. Cristhian Cabezas EXPERTO NACIONAL DE DESARROLLO	 Firmado electrónicamente por: CRISTHIAN FABIAN CABEZAS GUTIERREZ	
	Ing. Susana Samaniego Freire COORDINADOR NACIONAL CONTROL Y ASEGURAMIENTO DE CALIDAD	SUSANA ALEXANDRA SAMANIEGO FREIRE Firmado digitalmente por SUSANA ALEXANDRA SAMANIEGO FREIRE Fecha: 2024.08.05 16:45:28 -05'00'	
	Ing. Edison Cayambe Chicaiza COORDINADOR NACIONAL GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA	 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
ELABORADO POR:	Ing. Carmen Muñoz Ontaneda EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Firmado electrónicamente por: CARMEN ESTEFANIA MUNOZ ONTANEDA	

INFORMACIÓN DEL DOCUMENTO

Resumen:	En el presente documento se definen los estándares para codificación e implementación que deben cumplir las aplicaciones web a desarrollarse en la institución con el framework para front end denominado Vue.
Alcance:	Se define la utilización del framework Vue para la implementación del front end de las aplicaciones del SRI, así como también la utilización de HTML, CSS, JavaScript y la aplicación de conceptos de Responsive Design, Usabilidad y Accesibilidad Web.
Audiencia:	Personal de la Dirección Nacional de Tecnología

REVISIONES AL DOCUMENTO

Versión -Fecha	Autor	Descripción
1.0.0 / 2023-12-26	Carmen Muñoz	Emisión Inicial.
2.0.0 / 2024-08-05	Carmen Muñoz	Se agrega reglas para: • Uso de librerías para formateo de código. • Se actualiza reglas de nombrado de componentes. • Se actualiza estructura del proyecto.

Enmienda del documento

El Departamento de Planificación TI y Gestión de Información - Datos tiene la potestad de actualizar el contenido de los Anexos cuando lo considere necesario y no requerirá recolección de firmas para su aprobación. Sin embargo, se comunicarán a las Coordinaciones y Departamentos involucrados los cambios a realizarse, quienes podrán indicar sus observaciones al respecto en los siguientes ocho días laborables, luego de lo cual se considerarán aceptadas, y se procederá a su publicación.

La aplicación de las modificaciones al estándar en proyectos con desarrollos terminados o en curso será consensuado con el Departamento de Desarrollo, con el fin de evaluar el impacto (positivo o negativo) que tenga sobre el producto o sobre la calidad del mismo, adicionalmente se debe informar de los acuerdos a las Coordinaciones de Control y Aseguramiento de Calidad (QA) y Centro de Cómputo.

Ante cualquier sugerencia de mejora al documento debe ser comunicado al Departamento de Planificación TI y Gestión de Información - Datos para su análisis, evaluación de factibilidad y posterior actualización de la documentación en caso de considerarse necesario.

En caso de ser necesario emitir un lineamiento técnico, aclaración o alcance al presente documento, este podrá ser emitido mediante memorando suscrito por el Jefe Departamental de Planificación TI y Gestión de Información - Datos en coordinación con la Dirección Nacional de Tecnología; y se mantendrá como una disposición vigente hasta su incorporación en la siguiente actualización del documento.

Contenido

1.	Introducción	5
1.1.	Objetivo	5
2.	Estándares Generales	5
3.	Estructura y nombrado de proyectos	7
3.1.	Estructura de proyectos	7
3.2.	Nomenclatura	12
4.	Codificación	13
4.1.	Componentes UI	17
4.2.	Validación del lado del cliente (CSV)	19
4.3.	Pruebas unitarias	20
5.	Glosario	22
5.1.	Acrónimos	22
5.2.	Definiciones	22
6.	Bibliografía	22
7.	Anexos	23
7.1.	Anexo 1: Navegadores Web	23
7.2.	Anexo 2: Tiempos de carga y tamaño de página	23
7.3.	Anexo 3: Repositorio de fuentes de proyectos base	24
7.4.	Anexo 4: Componentes UI	24
7.5.	Anexo 5: Contenido archivo configuración prettier	25
7.6.	Anexo 6: Comandos para ejecutar eslint, prettier y pruebas unitarias (unit test)	25

1. Introducción

Contar en la institución con estándares de programación permite que el código de los sistemas institucionales que han sido desarrollados internamente sea legible, facilitando así el proceso de mantenimiento del código cuando se requiere agregar o modificar funcionalidades.

Con el objetivo de avanzar en la implementación de una nueva arquitectura resiliente para los sistemas institucionales y buscando brindar una mejor experiencia de usuario tanto para contribuyentes como usuarios internos, se ha definido que el front end de las aplicaciones institucionales sea desarrollado con el Framework JavaScript denominado VUE; por tal motivo surge la necesidad de estandarizar su programación para lograr que la misma, no solo sea legible, sino también homogénea permitiendo que todos los involucrados puedan entender el código implementado.

Es importante destacar que este documento no es una guía para la programación con VUE, para este fin, se encuentra disponible la documentación en el sitio oficial de este framework en las siguientes URLs: <https://vuejs.org/guide/introduction.html>, <https://vuejs.org/style-guide/>.

1.1. Objetivo

- Normar la programación del front end de las aplicaciones web institucionales que sean desarrolladas utilizando el framework VUE.

2. Estándares Generales

Regla: Los estándares definidos en este documento son de cumplimiento obligatorio en el desarrollo de aplicativos o componentes institucionales que hagan uso del Framework Vue para el desarrollo del front end de las aplicaciones. Para casos de excepción no descritos en este documento se deberá consultar y gestionar la aprobación a la Coordinación de Gestión de Datos y Arquitectura Tecnológica del Departamento de Planificación TI y Gestión de Información - Datos previa cualquier implementación.

Regla: El framework JavaScript a utilizar es Vue.

Utilidad: Vue es un framework progresivo que permite generar aplicaciones web SPA con un enfoque empresarial. Se puede combinar con otros frameworks JS. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: La suite de componentes UI a utilizar es PrimeVue.

Utilidad: PrimeVue es una librería JavaScript que posee una suite de componentes open source que es compatible con el framework Vue. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: La librería para el enrutamiento del lado del cliente a utilizar es vue-router.

Utilidad: vue-router es una librería oficial para enrutamiento en aplicaciones desarrolladas con Vue, que proporciona las herramientas necesarias para asignar y controlar los componentes de una aplicación a diferentes rutas de URL del navegador. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: La librería para el manejo de estados es Pinia.

Utilidad: Pinia es una librería para Vue, que permite compartir el estado entre componentes de la aplicación, de esta forma todos los componentes de la aplicación pueden acceder al valor de las

propiedades almacenadas en este almacenamiento (store). La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Nota: En Vue 2 se utiliza la librería Vuex para la gestión de estados, aunque es compatible con la versión de Vue 3, Pinia es una librería que tiene mejores prestaciones y es compatible con Vue3.

Regla: La librería para la interacción con API REST es Axios.

Utilidad: Axios es una herramienta compatible con la versión de Vue estandarizada, que permite realizar peticiones GET, POST, DELETE y otros para traer, enviar, eliminar y actualizar información desde un API; así también proporciona una API con funcionalidades tales como interceptores, cancelación de solicitudes y configuración global. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: Cuando se requiere hacer solicitudes http simples que no requieran manejar cancelaciones, interceptores o transformaciones de datos se puede usar fetch.

Utilidad: fetch es una API nativa del navegador para hacer solicitudes HTTP, no requiere de instalación de librerías y se considera óptima para solicitudes http simples que no requieran manejar cancelaciones, interceptores o transformaciones de datos complejas se puede usar fetch.

Regla: Las aplicaciones web expuestas en **INTERNET** deben ser RWD (desplegarse y utilizarse en dispositivos móviles), y las aplicaciones Web de **INTRANET** deben ser RWD si la misma requiere ser usada desde algún dispositivo móvil.

Utilidad: Asegurar, que el diseño de las aplicaciones esté optimizado para que las páginas se ajusten con relación al tamaño de la pantalla del dispositivo en el cual se accede a las aplicaciones.

Regla: La librería para el formateo de código es prettier.

Utilidad: Prettier es una herramienta de formateo de código que se utiliza para asegurar que el código siga un estilo consistente y predefinido en todo el proyecto. En las aplicaciones Vue 3, Prettier se usa junto con ESLint para mantener un código limpio, legible y estandarizado. Mientras que ESLint se enfoca en la calidad y corrección del código, Prettier se enfoca en el formato del código. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: Los aplicativos deben cumplir con las especificaciones indicadas en el documento **Manual de imagen para aplicaciones tecnológicas**, provisto por el Departamento de Comunicación Institucional.

Utilidad: Asegurar, que el diseño de todas las aplicaciones esté acorde a lo definido por el Departamento encargado de definir la imagen institucional.

Regla: Las aplicaciones web deben ser usables y accesibles. Para lo cual deben cumplir las reglas establecidas en las secciones: 4. Accesibilidad Web y 5. Usabilidad Web del documento **Estándar de Desarrollo – Aplicaciones Web**.

Utilidad: Permitir el acceso a las aplicaciones a la mayor cantidad de personas independientemente de sus limitaciones brindando una mejor experiencia de usuario.

Regla: Se debe verificar el funcionamiento de la aplicación mínimo en los navegadores y versiones especificadas en el [Anexo 1: Navegadores Web](#).

Regla: Se debe cumplir el tiempo de carga y pesos de las páginas web especificados en el [Anexo 2: Tiempo de carga y tamaño de páginas](#).

Regla: En cualquier nombrado en la codificación, se deben utilizar caracteres UTF8, (no incluir: tildes ni tampoco 'ñ' o caracteres especiales, salvo aquellos casos de funciones, eventos, etc que el framework VUE JS a usar así lo requiera).

Ejemplos:

- \$: existen varios usos, entre ellos cuando se requiere emitir eventos (\$emit), obtener las propiedades (\$props), obtener atributos (\$attrs) entre otros.
- #: se lo utiliza cuando se va a llamar a slots (<template #nombreSlot></template>)
- _: se usa guion bajo para indicar que ciertos elementos (métodos o propiedades) son privados o internos y no están destinados a ser accedidos directamente desde fuera del componente.

3. Estructura y nombrado de proyectos

3.1. Estructura de proyectos

Regla: Para la creación de una nueva aplicación web basada en el framework VUE se debe clonar el proyecto base (intranet o internet según corresponda) ubicado en el repositorio institucional (ver ubicación en el [Anexo 3: Repositorio de fuentes de proyectos base](#)), una vez clonado se debe renombrar la carpeta principal cambiando sri-base por el nombre correspondiente de acuerdo con lo indicado en el presente estándar.

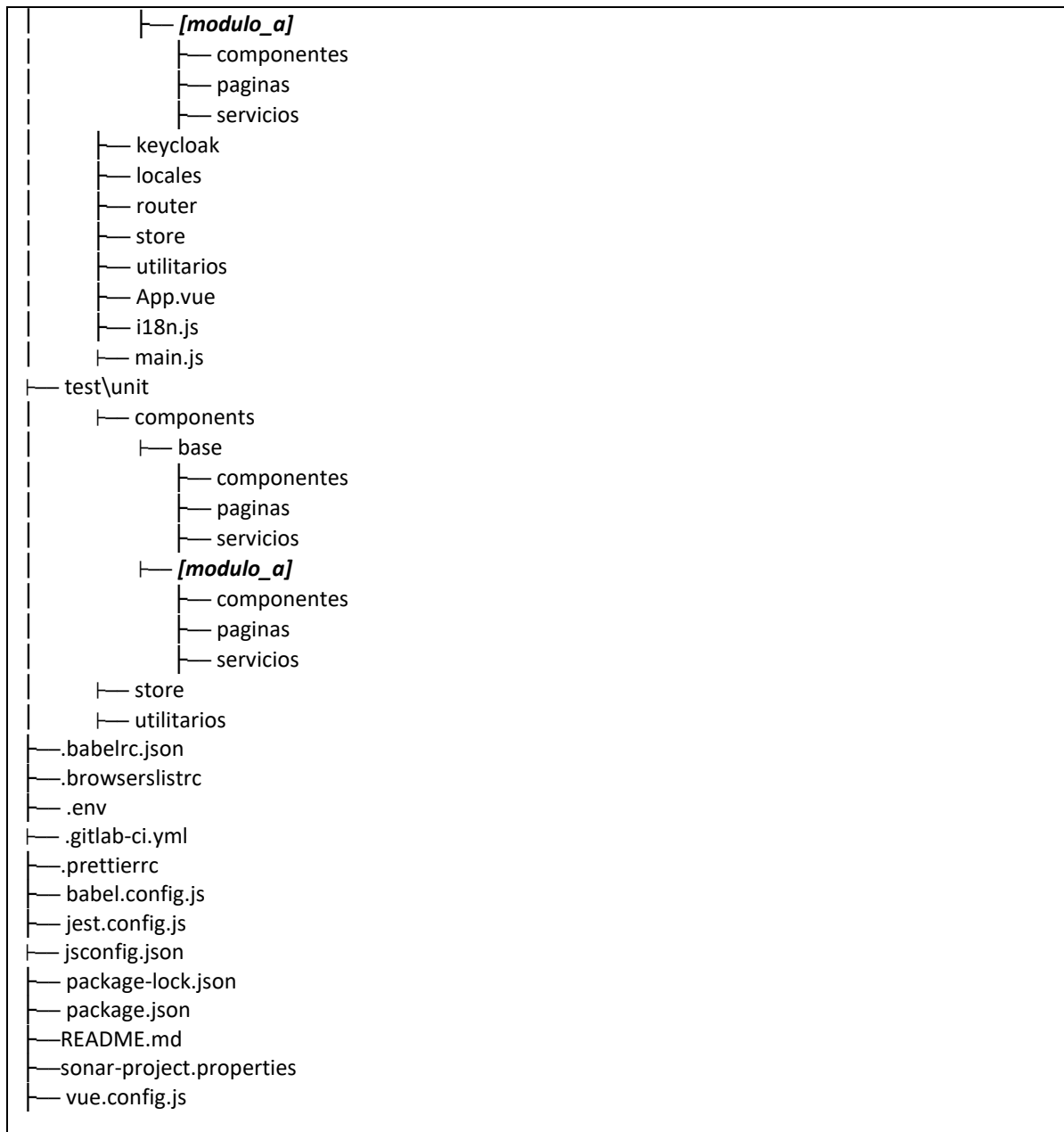
Utilidad: Los proyectos base fueron creados usando la herramienta CLI de Vue que genera la estructura de carpetas con todas librerías iniciales que se necesitan, así también, los proyectos base proveen el diseño con la funcionalidad mínima necesaria para: integración con Red Hat Single Sign On, generación de menú de opciones en base al perfil de usuario logueado, cumplimiento de estándares de accesibilidad web y manual de imagen para aplicaciones tecnológicas.

Regla: Tanto la estructura como los archivos del proyecto base no deben ser modificados sin previa autorización y justificación de la Coordinación de Gestión de Datos y Arquitectura Tecnológica del Departamento de Planificación TI y Gestión de la Información - Datos.

Utilidad: Mantener y respetar lo definido por la herramienta CLI de VUE en cuanto a la estructura base del proyecto y establecer las reglas para los componentes, páginas, ruteo, etc dentro de la carpeta src/components y las pruebas en test/unit/components.

Regla: La estructura del proyecto base es la siguiente:





El contenido de cada carpeta se detalla en la siguiente tabla:

Carpeta	Propósito
docker	Esta carpeta contiene el archivo Dockerfile para la construcción de la imagen de la aplicación en Docker.
kubernetes	Esta carpeta contiene los manifiestos para el despliegue del proyecto en la plataforma kubernetes.
public	Esta carpeta contiene 2 archivos: - favicon.ico: es el icono que aparece al lado del nombre de la pestaña del navegador. - public.html: es el archivo HTML sobre el que montará toda la página. No se debe modificar este archivo.
src\assets	En esta carpeta se encuentran todos los archivos estáticos que son específicos de la aplicación y que no son transversales, ya que éstos últimos deben estar en la carpeta de recursos centralizados en el servidor web.
src\components	En la carpeta src se sitúan los componentes de Vue y ficheros necesarios para que la aplicación funcione, los cuáles se encuentran en la carpeta components (en inglés debido a que se usó VUE CLI para la creación del proyecto), que es la carpeta principal que agrupa a todos los módulos que tendrá la aplicación web, inicialmente tendrá el módulo base, pero conforme se vayan agregando nueva funcionalidad se deberá agregar subcarpetas por cada módulo funcional que se vaya a incorporar.
src\components\base	Contiene toda la funcionalidad base para el funcionamiento de la aplicación web, por ejemplo: carga de menú de opciones, aplicación de imagen institucional, etc. Está dividido en tres subcarpetas: componentes, páginas y servicios.
src\components\base\componentes	Contiene todos los archivos de extensión “.vue” que son considerados como componentes reutilizables de la funcionalidad base de una aplicación web. ¹
src\components\base\paginas	Contiene todos los archivos de extensión “.vue” que pueden o no invocar a otros componentes o que tienen su funcionalidad específica.
src\components\base\servicios	Contiene los archivos de extensión .js que contienen las integraciones que se realizan con apis externas.
src\components\[modulo_a]	Por cada módulo funcional que tenga la aplicación, se deberá crear una carpeta con el nombre del módulo, por cada módulo se tendrá las subcarpetas: componentes, paginas y servicios.
keycloak	Contiene los archivos necesarios para la integración del aplicativo web con Red Hat Single Sign On.

¹ En las aplicaciones desarrolladas con el framework Vue JS, un componente es un archivo de extensión “.vue” que puede ser reutilizada desde una o varias páginas las veces que se lo requiera al incrustarlo dentro de la página como si fuera una etiqueta de HTML.

Carpeta	Propósito
locales	Contienen los archivos de extensión “.json” que contienen las etiquetas de la aplicación en el idioma que se requiere para la internacionalización de la aplicación web. Al momento se dispone de: es.json y en.json, para las etiquetas en español e inglés respectivamente. En estos archivos se debe configurar las etiquetas de todo el aplicativo para que funcione correctamente la internacionalización.
router	Contiene los archivos .js que se requieren para el ruteo (RuteoBase.js) y control de la autorización (AutorizacionGuardian.js) de la aplicación web. El archivo RuteoBase.js es el único que se debe modificar para incluir las rutas que se vayan necesitando conforme la aplicación vaya creciendo.
store	Contiene los archivos .js que representan los objetos que se requiere que sean accedidos por toda la aplicación. La librería que se usa para esto es Pinia.
utilitarios	Contiene archivos .js que contienen funciones que puede ser reutilizadas por toda la aplicación.
App.vue	Es el componente raíz de la aplicación y cumple las funciones de: punto de entrada de la aplicación, montaje de otros componentes, permite acceder a la instancia de Vue y sus propiedades desde cualquier lugar de la aplicación.
i18n.js	Componente que contiene la configuración para el cambio de idioma en las etiquetas de la aplicación.
main.js	Es el componente que se encarga de las siguientes tareas: configuración de Vue, importación, registro y montaje de componentes y arranque de la aplicación.
test\unit\components	Carpeta principal que agrupa las pruebas unitarias de la funcionalidad que tendrá la aplicación web, inicialmente tiene las pruebas unitarias del módulo base, pero conforme se vaya agregando nueva funcionalidad se deberá agregar subcarpetas con las pruebas unitarias respectivas agrupadas por sus módulos.
test\unit\components\ base	Carpeta que agrupa todas las pruebas unitarias de toda la funcionalidad base.
test\unit\components\ base\componentes	Contiene todos los archivos de extensión “.spec.js” que corresponden a las pruebas unitarias de los componentes reutilizables de la funcionalidad base.
test\unit\components\ base\paginas	Contiene todos los archivos de extensión “.spec.js” que corresponden a las pruebas unitarias de las páginas de la funcionalidad base.
test\unit\components\ base\servicios	Contiene todos los archivos de extensión “.spec.js” que corresponden a las pruebas unitarias de los servicios de la funcionalidad base.
test\unit\components\ base\utilitarios	Contiene todos los archivos de extensión “.spec.js” que corresponden a las pruebas unitarias de las funcionalidades de los archivos utilitarios.
test\unit\components\ [modulo_a]	Por cada módulo funcional que tenga la aplicación, se tendrá la carpeta que agrupa todas las pruebas unitarias de cada módulo y por cada módulo se tendrá las subcarpetas: componentes, paginas, servicios.

Carpeta	Propósito
.babelrc.json	Archivo generado automáticamente con la creación del proyecto base al usar VUE CLI, se utiliza para especificar la configuración de Babel ² .
.browserslistrc	Archivo generado automáticamente con la creación del proyecto base al usar VUE CLI, se utiliza para definir los navegadores y las versiones de los navegadores que soporta la aplicación.
.env	Contiene las configuraciones de las variables de entorno a ser utilizadas por la aplicación.
.prettierrc	Archivo que define las reglas y configuraciones específicas que la librería para formateo de código (Prettier) debe aplicar al dar formato al código JavaScript y Vue.
.gitlab-ci.yml	Archivo de configuración utilizado en GitLab para definir el proceso de integración continua (CI) y despliegue continuo (CD). En este archivo se especifican los trabajos (jobs) y etapas (stages) que GitLab CI/CD debe ejecutar automáticamente cuando ocurren ciertos eventos en el repositorio, como commits, merges, o la creación de tags. A la fecha de elaboración de este documento se encuentran configuradas los siguientes grupos de tareas build, test, package, deploy, testqa y deploy-pro. Dichos grupos contienen tareas que permiten la compilación, análisis de código, pruebas, generación de la imagen de contenedor y despliegue en una plataforma de contenedores.
babel.config.js	Archivo que contiene las configuraciones de babel, la herramienta que transpila el Javascript generado a código entendible por navegadores antiguos. No debe ser modificado.
jest.config.js	Contiene las configuraciones de la librería para pruebas unitarias jest.
jsconfig.json	Archivo que contiene la configuración de TypeScript para aplicaciones Vue que utilizan Jest y Vue 3 Jest para pruebas unitarias. El proyecto base no contiene código TypeScript, se deja el archivo para el futuro si algún proyecto decide implementar su código con TypeScript en lugar de JavaScript.
package-lock.json	Archivo que se crea automáticamente, cuyo objetivo es que se bloquee la versión con la que fue instalada una dependencia. No debe ser modificado.
package.json	Archivo que contiene información acerca del proyecto (nombre, versión, etc.) y posee el listado de los paquetes o librerías de los que depende con su versión.
README.md	Contiene la explicación de lo que el proyecto base contiene y la forma de compilarlo y desplegarlo. Una vez que se adecúe para un proyecto específico debe ser modificado.

² Babel es un transpilador de JavaScript que convierte el código ES6+ (o cualquier versión más reciente de ECMAScript) en una versión compatible con navegadores más antiguos que no soportan estas características de forma nativa.

Carpeta	Propósito
sonar-project.properties	Archivo de configuración que permite que la herramienta Sonarqube realice la revisión de código estático de la aplicación, este archivo solo puede ser modificado por la Coordinación de Control de Calidad.
vue.config.js	Archivo de configuración centralizado de los proyectos Vue, que permite definir y modificar varias configuraciones que afectan al proyecto y que están relacionadas con el proceso de compilación, contexto de publicación, título de la aplicación, internacionalización y plugins que se requieren para el desarrollo (ejemplo: procesadores de HTML o de CSS). Al momento se encuentran configuradas las opciones para la internacionalización y el contexto para la publicación.

3.2. Nomenclatura

Regla: Los nombres de los componentes, páginas, utilitarios y servicios deben ser en español, sin tildes, ñ, caracteres especiales, deben tener al menos dos palabras y estar en notación PascalCase.

Ejemplo

Nombre de Componente

CabeceraGeneral.vue

Nombre de Página

InicioBase.vue

Nombre de Servicio

AutorizacionServicio.js

Regla: Los componentes vue en el contenido HTML deben estar en notación kebab-case, mientras que en el contenido JavaScript deben estar en notación PascalCase.

Utilidad: Tener código limpio y fácil de lectura.

Ejemplo

```
<!-- kebab-case -->
<template>
  <cabecera-general />
  <modulo-base />
</template>

<!-- PascalCase -->
<script>
  import CabeceraBase from '@components/base/componentes/CabeceraBase.vue'
  import ModuloBase from '@components/base/componentes/ModuloBase.vue'

  export default {
    components: {
      CabeceraBase,
      ModuloBase
    }
  }
</script>
```

Regla: El nombre de los servicios debe tener como sufijo la palabra servicio y se debe crear un archivo por cada diferente api con la cual interactúe la aplicación.

Utilidad: Identificar con facilidad los archivos cuya implementación se utilizan para integraciones con apis externas.

Ejemplo

Nombre de servicio

AutorizacionServicio.js
CatastroTributarioServicio.js

Regla: Los objetos cuya información requiera ser accedida o modificada desde cualquier módulo de la aplicación deben ser creados dentro la carpeta **src/store**. Su nombre debe ser en singular y en PascalCase y debe tener el sufijo Store.

Utilidad: Identificar con claridad los objetos cuyo estado puede ser accedido o modificado por toda la aplicación

Ejemplo

Nombre de store

UsuarioStore.js
ParametroGeneralStore.js

Nota: El archivo UsuarioStore.js es parte del proyecto base por lo que no puede ser modificado sin la aprobación de la Coordinación de Gestión de Datos y Arquitectura.

4. Codificación

Regla: Todos archivos fuente (componentes, páginas, servicios, utilitarios), deben tener al inicio un comentario en el cual se debe especificar:

- Nombre, debe incluir la extensión de este.
- Descripción
- Autor
- Copyright

Ejemplo

Archivos extensión .vue

```
<!--
* Nombre: InicioBase.vue
* Descripción: página principal del proyecto base intranet en Vue que orquesta la presentación de todos los componentes.
* Autor: Planificación TI y Gestión de Información - Datos
* Copyright 2023 Servicio de Rentas Internas.
-->
```

Archivos extensión .js

```
/*
* Nombre: AutorizacionServicio.js
* Descripción: servicio para interactuar con el api rest que consulta la información de autorización de un
* Autor: Planificación TI y Gestión de Información - Datos
* Copyright 2023 Servicio de Rentas Internas.
```

*/
Archivos extensión .css
/* * Nombre: errores.css * Descripción: Hoja de estilos de páginas de errores para servidores. * Autor: Planificación TI y Gestión de Información - Datos * Copyright 2023 Servicio de Rentas Internas. */

Nota: Archivos de terceros (por ejemplo, scripts) deben mantener sus comentarios y en el caso de no tenerlos no es obligatorio colocar el comentario.

Regla: El formato en pantalla para las fechas es: dd/MM/yyyy, es decir, día, mes y año. De la misma manera el formato para las fechas que incluyan valores de tiempo debe ser el siguiente dd/MM/yyyy HH:mm:ss.

Regla: El formato en pantalla para los valores numéricos debe regirse a lo indicado en el manual de imagen para aplicaciones tecnológicas.

Regla: El contenido de los archivos fuentes de extensión .vue deben estar organizados en las siguientes secciones:

- Comentario del archivo (revisar regla de estándar que debe cumplir)
- Contenido HTML
- Contenido JavaScript
- Contenido CSS: no está permitido el uso de css específicos por cada componente, se deben usar los archivos de recursos centralizados o los que se encuentran en: src\components\assets.

Ejemplo

```

<!--
* Nombre: InicioBase.vue
* Descripción: página principal del proyecto base intranet en Vue que orquesta la presentación de todos los
componentes.
* Autor: Planificación TI y Gestión de Información - Datos
* Copyright 2023 Servicio de Rentas Internas.
-->
<template>
  <!-- Contenido HTML-->
</template>

<script>
  /** Contenido JavaScript**/
</script>

```

Utilidad: Con esta organización los archivos de extensión .vue serán más legibles para los desarrolladores.

Regla: El lenguaje de programación que se utiliza tanto en los archivos .vue como para los servicios y utilitarios es JavaScript o TypeScript.

Regla: El código de los archivos fuentes, debe estar indentado, la misma será utilizando espacios en blanco, también se puede usar tabulaciones.

Utilidad: Código fuente más legible y mantenible.

Regla: Los nombres de: variables del componente (data), atributos (props), funciones (methods, computed, watch, emit), deben ser escritos en español, sin tildes, ñ, caracteres especiales; si es una palabra simple deberá estar en minúsculas, caso contrario deben cumplir con el estándar lowerCamelCase.

Ejemplo

```
<script>
export default {
  data() {
    return{
      /*Variables del componente*/
      usuarioLogueado: null,
      modulosArreglo: [],
    }
  },
  /*Atributos*/
  props: {
    autorLibro: {
      type: String,
      default: "",
      required: true
    }
  },
  /*Propiedades computadas*/
  computed: {
    calcularIVA() {
      return valor*1,12;
    }
  }
  /*Metodos*/
  methods:{
    obtenerInformacionUsuario(codigoAplicacion) {
      ...
    },
    cerrarSesion () {
      ...
    },
  },
  /*observadores*/
  watch: {
    calcularRango(valor){
      ...
    }
  }
}
</script>
```

Regla: Los nombres de funciones (methods, computed, watch, emit) deben ser descriptivos y escritos en infinitivo indicando así la acción que van a realizar cuando son ejecutados.

Regla: Las variables de componentes (data) deben ser inicializados al momento de su declaración.

Ejemplo

```
<script>
export default {
  data() {
    return{
      usuario: null,
```



```
modulosArreglo: [],  
nombreAplicacion:"",  
mostrarMenu: true,  
menusExpandidos: {},  
tiempoInactividadUsuario: constantesAplicacion.TIEMPO_INACTIVIDAD_USUARIO,  
}  
},  
</script>
```

Regla: Los props deber ser inicializados con los siguientes atributos: type, default y required; el atributo validator es opcional.

Ejemplo

```
<script>  
export default {  
  props: {  
    autorLibro: {  
      type: String,  
      default: "",  
      required: true  
    }  
  }  
}  
</script>
```

Regla: Cuando se utilice la directiva v-for, se debe incluir el atributo key con una clave única por cada elemento que se vaya a iterar.

Utilidad: Ayudar a Vue a optimizar el renderizado del DOM cuando se realizan actualizaciones.

Ejemplo

```
<ul>  
  <li  
    v-for="todo in todos"  
    :key="todo.id"  
  >  
    {{ todo.text }}  
  </li>  
</ul>
```

Regla: Nunca se debe usar las directivas v-if y v-for en el mismo elemento.

Utilidad: Cuando Vue procesa las directivas, v-if tiene mayor prioridad que v-for, por lo que podría arrojar un error o generarse resultados no válidos si la variable sobre la cual se está iterando es nula o no ha sido asignada con el valor correcto.

Ejemplo

Incorrecto	Correcto	
	Opción 1	Opción 2
<pre> v-for="usuario in usuarios" v-if="usuario.isActive" :key="usuario.id" > {{ usuario.name }} </pre>	<pre> v-for="usuario in usuariosActivos" :key="usuario.id" > {{ usuario.name }} </pre> * En esta opción se itera sobre una lista de usuarios activos que fue previamente filtrada.	<pre> <template v-for="usuario in usuarios" :key="usuario.id"> <li v-if="usuario.isActive"> {{ usuario.name }} </template> </pre> * En esta opción se coloca el v-for y el v-if en elementos html diferentes.

4.1. Componentes UI

Regla: Todo componente UI de la suite de PrimeVue debe tener configurado el atributo id.

Regla: El atributo id de los componentes UI debe ser un nombre descriptivo con un prefijo que permita identificar el tipo de componente, el prefijo a ser usado por cada tipo componente se encuentra definido en [Anexo 4: Componentes UI](#).

Nota: En el [Anexo 4: Componentes UI](#) se ha incluido un listado de componentes con sus prefijos, en caso de requerir un componente UI que no se encuentre dentro de lo indicado en este anexo, se debe solicitar su inclusión a la Coordinación de Gestión de Datos y Arquitectura Tecnológica a través de un requerimiento.

Regla: Cuando se incluyan componentes de tipo file upload se debe especificar el valor del peso del archivo y el número de archivos a cargar. Por defecto el número máximo de archivos a cargar es de 1 y el valor total de la carga no debe exceder los 10 megabytes. De ser necesario el cambio del valor total de la carga se debe contar con la autorización de la Coordinación de Gestión de Datos y Arquitectura Tecnológica.

Nota: El tiempo de carga de un archivo depende de la velocidad de subida de archivos de los clientes³ y del tiempo máximo de espera para la carga es de 2 minutos, teniendo en cuenta lo anterior se determinó que los archivos deben tener un máximo de 10 megabytes, ya que con eso no se sobrepasa el tiempo de la sesión según las distintas velocidades de subida que se pueden contratar en nuestro país. Calculado según [Databarracks, Bandwidth Calculator, 2022]

Velocidad de Subida	Tiempo de subida de paquete de 10MBytes
1 Mbps	1 minuto, 23 segundos
2 Mbps	41 segundos
10 Mbps	8 segundos
100 Mbps	Menos de 1 segundo

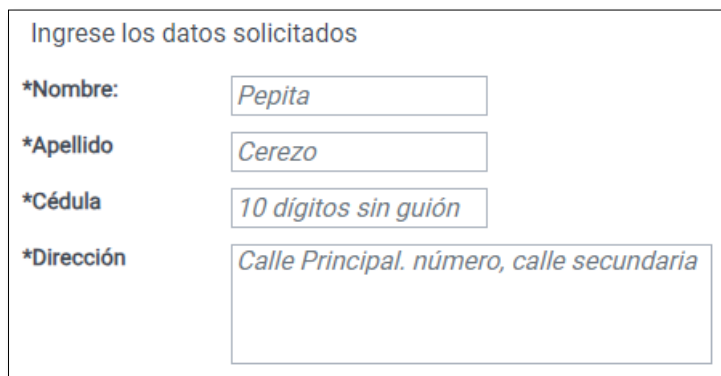
Regla: Los valores de las etiquetas y títulos de componente UI como: botones, labels, enlaces, formularios, páginas, agrupadores de componentes, mensajes de alerta deben escribirse iniciando con la primera letra en mayúscula.

Nota: Los mensajes de alertas que se presentan en las pantallas son definidos por los responsables funcionales de cada aplicación.

³ En Ecuador según sitio speedtest.net a septiembre 2023 la velocidad promedio de subida es de 12.37Mbps en móviles y 62.94Mbps en internet fijo (<https://www.speedtest.net/global-index/ecuador>).

Regla: Todo componente UI que implique ingreso de texto debe definir un valor en el atributo placeholder que describa brevemente el valor o el formato que se espera recibir sin que supere el ancho del control y pierda visibilidad.

Ejemplo:



Formulario de ejemplo con el título "Ingrese los datos solicitados". Contiene cuatro campos de texto con los siguientes placeholders:

- *Nombre: *Pepita*
- *Apellido: *Cerezo*
- *Cédula: *10 dígitos sin guión*
- *Dirección: *Calle Principal. número, calle secundaria*

Regla: Cuando se necesite proveer de una descripción más detallada, se debe utilizar un tooltip o atributo title. Si el componente UI no dispone de este atributo se debe definir un tooltip/title accedido mediante un ícono de ayuda colocado junto al componente o etiqueta de este.

Regla: Un componente de tipo botón que necesite mostrar una descripción más detallada de su funcionamiento debe definir un atributo title o tooltip que se muestra al colocar el puntero del mouse sobre el mismo.

Regla: El contenido de los tooltips y/o atributos title deben seguir los siguientes lineamientos:

- No debe ser mayor a 50 caracteres incluyendo espacios de separación.
- No debe contener texto rico es decir con formatos especiales en parte de este ni tampoco imágenes.
- No se debe incluir caracteres de avance o retroceso de línea.

Utilidad: Facilita la lectura del texto complementario sin sacrificar peso de página, ni generar confusiones al lector.

Regla: Los valores de los componentes de tipo lista deben contener mínimo un identificador único y una etiqueta.

Utilidad: Que todos los elementos de la lista no tengan identificadores y/o valores repetidos o nulos.

Regla: Todo control de tipo Text Área debe establecer por defecto una altura menor o igual a 4 líneas visibles.

Regla: Toda tabla deben cumplir las siguientes condiciones:

- **Total de registros:** Deben mostrar el número total de registros de la tabla, conforme lo indique la Guía de Imagen Institucional. Para el caso de ausencia de registros debe mostrar un mensaje indicando la falta de registros.
- **Scrollbar:** En las tablas se muestran solo los campos principales con el fin de evitar el uso del scrollbar. Los contenidos de las columnas deben mostrar lo estrictamente necesario.
- **Número de columnas:** Las tablas no deberán mostrar una cantidad mayor a 10 columnas (incluidas las columnas de acción como: seleccionar, modificar, eliminar, ver, si existieran). Siempre tomando en cuenta los temas usabilidad.

- **Nombre de columnas:** El título de las columnas deben ser autodescriptivos y no extensos. Hay que optimizar lo máximo posible, los títulos formados con más de tres palabras no deben ser incluidos, sino más bien optimizarlos a solo tres palabras. Y de ser posible utilizar abreviaciones bien conocidas, como RUC, PVP, USD, etc., estas abreviaciones deben indicar su significado a través del atributo title o un tooltip. Ejemplo: Si el título de la columna es Valor Total Presupuestado por Mes, debe ser optimizado a máximo tres palabras: Valor Mensual Presupuestado.
- **Selección múltiple:** Para selecciones múltiples se usa el componente UI checkbox colocado como primera columna. Debe existir un checkbox global para poder seleccionar todos los registros.
- **Filtros:** No usar los filtros de las cabeceras de las columnas, en su lugar debe existir una sección externa de filtros, con al menos un campo requerido para que realice la búsqueda.
- **Carga de data por defecto:** Mostrar un máximo de 10 registros, y su respectiva sección de paginación.
- **Ordenamiento de datos:**
 - La información que se muestra en las tablas u otros componentes debe venir ordenada desde la base de datos según el requerimiento propio del negocio, o en casos excepcionales por rendimiento el ordenamiento se lo deberá hacer a nivel del servicio.
 - Se podrá implementar mecanismos de ordenamiento que puedan ser usados por el usuario en tiempo de ejecución (por ejemplo, el uso de “sort by” en las “datatable”) siempre y cuando no ocasionen un deterioro del rendimiento de la página.
- **Tablas con más de 10 registros:** Debe tener una opción para que el usuario cambie el número de registros que se muestran en cada página de la tabla, los valores a mostrar son: 10, 20, 30, 40 y 50.
- **Tablas con más de 100 registros:**
 - No mostrar información precargada por defecto.
 - **Paginación:** Utilizar la paginación visual de la tabla y cargar la data de cada página bajo demanda. Esto quiere decir, que, según el contexto, no bastará con especificar un componente de paginación a nivel de la página web, sino que será necesario implementar un mecanismo de paginación programático sobre las consultas sobre el servicio que se utilice.

Regla: Las etiquetas (texto) para los botones no deben ser superiores a tres palabras.

Regla: El contenido de los componentes Tab (Tabpanel, TabView) debe cargarse por demanda al realizar clic por primera vez en cada una de las solapas o tabs.

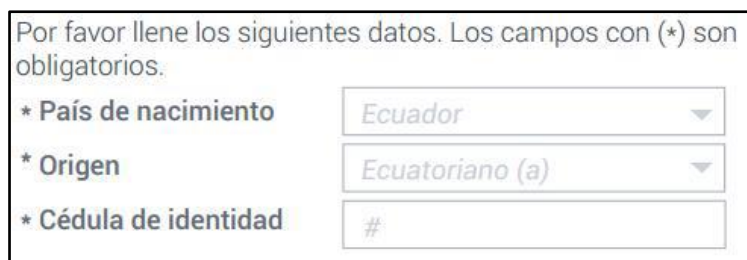
4.2. Validación del lado del cliente (CSV)

Regla: Las validaciones establecidas del lado del cliente son obligatorias y no sustituyen a las validaciones que deben efectuarse del lado del servidor, aun cuando estas sean iguales.

Regla: Las validaciones que deben tener los campos de los formularios deben corresponder a:

- Validación de valores requeridos.
- Validación de formato de los valores ingresados.
- Validación de datos de confirmación.
- Validación de caracteres específicos de los campos según el formato de datos ingresados.

Regla: Los campos sobre los cuales se aplique una validación de campo obligatorio deben estar plenamente identificados. Ejemplo:



Por favor llene los siguientes datos. Los campos con (*) son obligatorios.

* País de nacimiento	Ecuador
* Origen	Ecuatoriano (a)
* Cédula de identidad	#

Figura 1: Ejemplo de formulario con campos requeridos.

Regla: El resultado de la validación de él o los campos deben retornar la respectiva información visual de que la misma se cumplió o caso contrario el mensaje con la información del error.

Regla: Los mensajes de error deben ser descriptivos sin que superen los 200 caracteres.

Regla: Los mensajes de error devueltos por el proceso de validación del lado del servidor deben mostrarse en la sección de visualización de errores agrupados en una alerta de error.

Regla: Ningún proceso de validación debe mostrar otra página ni ventanas emergentes con el fin de mostrar el resultado del proceso de validación.

Regla: Los componentes UI que son utilizados para ingresar texto como por ejemplo las cajas de texto y text áreas deben contemplar la validación de restricción de longitud del texto (longitud mínima y máxima) que se ingresa, la longitud máxima debe ser menor o igual al tamaño del campo correspondiente en la tabla de la base de datos donde este finalmente pueda ser almacenado.

4.3. Pruebas unitarias

Regla: La librería para el desarrollo de pruebas unitarias es Jest.

Utilidad: Jest es una librería especializada para el desarrollo de pruebas unitarias y es compatible con la versión estandarizada de Vue. La versión a utilizar se indicará en la Arquitectura de la Solución Tecnológica de cada aplicación.

Regla: El porcentaje de cobertura del código debe respetar lo definido en el Procedimiento de Desarrollo y Mantenimiento de Aplicativos Informáticos Institucionales, sección 3.1.8 “LA CALIDAD” DURANTE EL PROCESO DE DESARROLLO, literal “g”.

Regla: El nombre de las pruebas unitarias debe ser claro y descriptivo, e iniciará con la palabra “Deberia”.

Utilidad: Los nombres de las pruebas al ser claros y descriptivos ayudar a entender fácilmente lo que se está probando.

Ejemplo

```
test( 'Deberia ocultar menu extendido cuando este oculto', () => {  
  wrapperPrincipal.vm.mostrarMenu = true  
  wrapperPrincipal.vm.mostrarOcultarMenu()  
  expect( wrapperPrincipal.vm.mostrarMenu).toBe(false)  
})
```

Regla: Las pruebas unitarias deben cumplir con el patrón Arrange (Preparar) – Act (Actuar) – Assert (Afirmar) conocido también como AAA.

Utilidad: El patrón AAA ayuda a que la prueba unitaria sea ordenada, fácil de entender y dar mantenimiento. Para el caso de Vue:

- *Arrange*: En esta fase, se preparan los datos y el entorno necesarios para la prueba. Esto puede incluir la creación de un componente Vue, la configuración de sus propiedades y métodos, y la simulación de eventos.
- *Act*: En esta fase, se ejecuta el código que se está probando. Esto puede incluir llamar a un método, llamar a una función o realizar una operación.
- *Assert*: En esta fase, se verifican los resultados de la prueba. Esto se hace usando afirmaciones, que son funciones que verifican que el resultado de la prueba sea el esperado.

Regla: Para definir pruebas unitarias individuales se debe usar la función **test**.

Ejemplo

```
test('Debería renderizar el texto correcto', () => {
  const wrapper = mount(MyComponent, {
    propsData: {
      text: 'Hola mundo!',
    },
  });
  expect(wrapper.text()).toBe('Hola, mundo!');
});
```

Regla: Para definir un grupo de pruebas unitarias se debe usar la función **describe**.

Ejemplo

```
describe('MyComponent', () => {
  it('Debería renderizar el texto correcto ', () => {
    const wrapper = mount(MyComponent, {
      propsData: {
        text: 'Hola, mundo!',
      },
    });
    expect(wrapper.text()).toBe('Hello, world!');
  });

  it('Debería llamar a método onMounted', () => {
    const wrapper = mount(MyComponent);
    expect(wrapper.vm.$onMounted).toHaveBeenCalled();
  });
});
```

Regla: Las pruebas unitarias deben usar mocks cuando el código que se está probando utiliza dependencias externas.

5. Glosario

5.1. Acrónimos

API: Application programming interface o interfaz de programación de aplicaciones.

SPA: Single Page Application

RWD: Responsive Web Desing o Diseño Web Responsivo

5.2. Definiciones

API – Interfaz de Programación de Aplicaciones: son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos.

Diseño Web Responsivo: una forma de desarrollar sitios web asegurando, que el diseño está optimizado para que el formato se ajuste con relación al tamaño de la pantalla del dispositivo

Indentación: significa mover un bloque de código a la derecha utilizando espacios en blanco.

Single Page Application: Una aplicación de una sola página, es un tipo de aplicación web que carga una única página inicial desde el servidor y luego actualiza dinámicamente su contenido, mediante las interacciones del usuario, sin necesidad de cargar páginas adicionales del servidor.

6. Bibliografía

- Amazon. (2023, Noviembre 17). *AWS* *Amazon.com*. From <https://aws.amazon.com/es/what-is/api/>
- Databarracks. (2024, Noviembre 17). *Calculadora de ancho de banda*. From <http://www.databarracks.com/support/bandwidth-calculator/>
- *Guía Oficial de Vue*. (n.d.). From Vue.js: <https://vuejs.org/guide/introduction.html>
- Hubspot.es. (2023). *Qué es una single page application, cómo funciona y ejemplo*. From [https://blog.hubspot.es/website/que-es-single-page-application#:~:text=Una%20single%20page%20application%20\(SPA,cargar%20p%C3%A1ginas%20adicionales%20del%20servidor.](https://blog.hubspot.es/website/que-es-single-page-application#:~:text=Una%20single%20page%20application%20(SPA,cargar%20p%C3%A1ginas%20adicionales%20del%20servidor.)
- rgrandagarcia. (2012, Octubre 2). *Más que codificar*. From <https://masquecodificar.wordpress.com/2012/10/02/disenio-web-responsivo-rwd/>
- Vue JS. (2023, Noviembre 17). *Vue - El Marco Progresivo de JavaScript*. From <https://vuejs.org/>
- *Vue Style Guide*. (2024, 01 03). From <https://vuejs.org/style-guide/>

7. Anexos

7.1. Anexo 1: Navegadores Web

Tecnología	Versión	Ambiente
Microsoft Edge	101 o superior	Internet
Firefox	100 o superior	Internet/Intranet
Chrome	102 o superior	Internet/Intranet

7.2. Anexo 2: Tiempos de carga y tamaño de página

El tiempo de descarga de cualquier página web que pertenezca a un sistema, no deberá sobrepasar la media de un segundo y hasta un máximo de 5 segundos. Todas las páginas deberían cumplir este límite de tiempo especificado; sin embargo, en casos excepcionales de páginas con contenidos tabulares y/o resultados de procesamiento el tiempo límite permitido es de hasta 8 segundos para todo tipo de dispositivo.

Nota: Según Jakob Nielsen⁴, existen 3 tipos de tiempo de respuesta establecidos:

- **0,1 segundos:** es el límite para que el usuario sienta que el sistema reacciona de forma instantánea, es decir, que manipula directamente los datos desde la interfaz.
- **1 segundos:** es el límite para que los usuarios sientan que el equipo está trabajando, en lugar de ser un efecto directo sobre las acciones que se requiere.
- **10 segundos:** es el límite para mantener la atención del usuario en la tarea. Cualquier actividad más lenta de 10 segundos, requiere una optimización de la aplicación ya que, los usuarios desearán realizar otras tareas mientras se espera el equipo finalice su actividad.

El **peso de las páginas** debe ser validadas con compresión y sin habilitar el cache en función de un tamaño máximo y un tamaño máximo excepcional debidamente justificado. La siguiente tabla muestra los tamaños permitidos para las páginas web.

Tecnología o Framework	Tamaño Máximo en KB	Tamaño Máximo Excepcional en KB
HTML5, JS, CSS3 (Framework JS)	990 y 60*	1124 y 100*

* Segunda carga con caché y compresión.

Utilidad: Esto permite que en los desarrollos se busque la optimización de los recursos web. Por lo general el texto siempre es lo primero que se descarga, luego las imágenes y otro tipo de archivos más pesados. El tiempo total de descarga del archivo dependerá de la relación PESO - VELOCIDAD de transferencia. Esta es una relación proporcional en la que hay que tener en cuenta, que equipo y conexión posee el target.

Nota: Para todos los casos se debe verificar si la herramienta de medición que se utilice muestre correctamente los archivos con compresión ya que se ha encontrado casos donde la herramienta no muestra que los archivos estén comprimidos, en ese caso para la verificación se debe utilizar herramientas alternativas.

Medición de peso de página y tiempo de carga, la medición del peso y tiempo de carga de la página se realiza mediante la herramienta de desarrollador de los navegadores Chrome o Firefox deshabilitando el caché y verificando que los archivos se muestren con compresión, y solo en los casos que se considere necesario se verifica mediante Y-slow, complemento que se instala en los navegadores (Firefox y Chrome).

7.3. Anexo 3: Repositorio de fuentes de proyectos base

Ambiente	Ubicación Alfresco
Intranet	https://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-base

7.4. Anexo 4: Componentes UI

La suite de componentes UI a utilizar es PrimeVue, a continuación, se definen los prefijos para el nombrado de los componentes que se pueden utilizar para el desarrollo de aplicaciones web, en caso de requerir un componente UI que no se encuentre dentro la siguiente tabla se debe solicitar su inclusión a la Coordinación de Gestión de Datos y Arquitectura Tecnológica a través de un requerimiento.

COMPONENTE UI	PREFIJO
Button	btn
Breadcrumb	bcb
Calendar	cal
Column	clm
DataTable	dtl
Dialog	dlg
Dropdown	ddn
Fieldset	fdt
InlineMessage	ims
InputText	txt
Message	msg
Panelmenu	pmn
RadioButton	rbn
Steps	stp
TabPanel	tpl
TabMenu	tmn
TabView	tvw
Textarea	txa
Toast	tst
Tooltip	tol

7.5. Anexo 5: Contenido archivo configuración prettier

Archivo .prettierrc

```
{  
  "singleQuote": true,  
  "tabWidth": 4,  
  "printWidth": 100,  
  "semi": false,  
  "trailingComma": "none"  
}
```

7.6. Anexo 6: Comandos para ejecutar eslint, prettier y pruebas unitarias (unit test)

Librería	Comando
Eslint (revisión de código estático)	npm run lint
Prettier	npm run format
Pruebas Unitarias (Unit Test)	npm run test:unit